

제 4 주:

가방 성능비교



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



[제 4 주] 실습 1

가방의 성능 측정

□ 실습 목표

■ 이론적 관점

- 리스트의 다양한 구현 방법에 따른 차이점
 - ◆ 기능 별 장단점을 이해한다.
- 프로그램의 성능 측정 방법

■ 구현적 관점

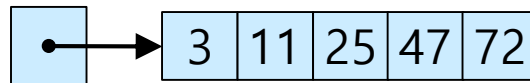
- 다양한 리스트를 구현
- 리스트의 각 구현에 따른 성능을 측정한다.
- 실제 측정 값들이 이론적 예측과 일치하는지 확인한다.

과제에서 해결할 문제



문제 [1]

- 가방의 구현에 따른 성능을 측정한다.
 - 먼저 Sorted Array Bag에 대한 성능 측정을 수행하여 본다.
 - ◆ Sorted Array Bag(Increasing order)



- 성능 측정을 할 행위
 - ◆ 데이터의 삽입
 - ◆ 최대값 검색

□ 문제 [1]

■ 입출력

- 입력:
 - ◆ 없음
 - ◆ 필요한 데이터는 프로그램에서 생성
- 출력: 성능 측정 결과:
 - ◆ 데이터 크기 변화에 따른 성능 측정 결과
 - 10000, 20000, 30000, 40000, 50000

□ 출력의 예

<<List의 구현에 따른 실행 성능 차이 알아보기>>

[Sorted Array List]

크기: 1000	삽입하기: 4135656	최대값찾기: 107712
크기: 2000	삽입하기: 12332793	최대값찾기: 153608
크기: 3000	삽입하기: 3784625	최대값찾기: 224874
크기: 4000	삽입하기: 6692358	최대값찾기: 303500
크기: 5000	삽입하기: 10360617	최대값찾기: 322748

수행시간 측정 방법



□ 시간 비교

■ System.nanoTime()

- 자바에서 현재 시간을 얻어오는 시스템 메소드
- 단위 : nano(=10⁻⁹) second
- Long 타입으로 저장
- 예시

```
long startTime = System.nanoTime();
```

```
함수 실행();
```

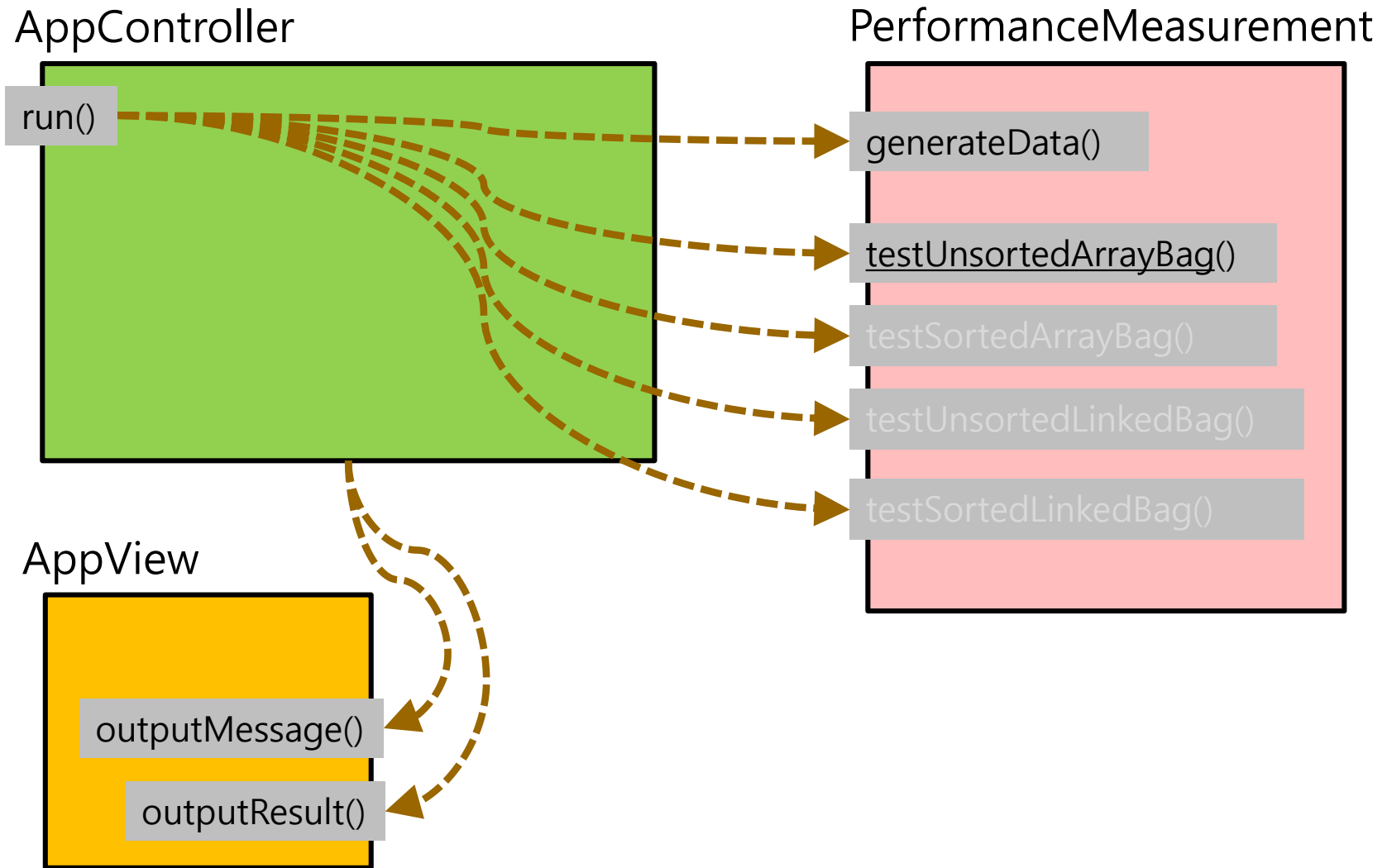
```
long stopTime = System.nanoTime();
```

```
long duration= (stopTime - startTime);
```

```
System.out.println(" 수행 시간 = " + duration + " nano sec");
```

- 자료의 입력 크기가 작을 경우 현재 컴퓨터의 성능이 좋아 millisecond 단위로는 확인이 불가능하여 JDK 1.5 이후 추가됨

이 과제에서 필요한 객체는?



이 과제에서 필요한 객체는?

- ApplicationController
- AppView

- Model
 - PerformanceMeasurement
 - TestResult
 - UnsortedArrayBag <E>
 - SortedArrayBag
 - UnsortedLinkedBag
 - SortedLinkedBag
 - Coin
 - Node

main()

□ main()을 위한 class

```
public class DS1_04_학번_이름 {  
    public static void main (String[] args)  
    {  
        ApplicationController appController = new ApplicationController() ;  
        // ApplicationController 가 실질적인 main class 이다.  
        appController.run() ;  
        // 여기 main()에서는 앱 실행이 시작되도록 해주는 일이 전부이다.  
    }  
}
```

Class "AppController"



□ Class "AppController" 의 구현 [1]

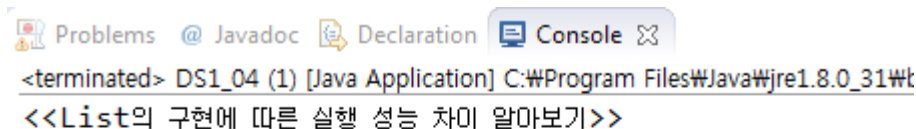
```
public class AppController {  
    // 비공개 변수들  
    private AppView _appView ;  
    private PerformanceMeasurement _pm;  
  
    // 생성자  
    public AppController()  
    {  
        this._appView = new AppView() ;  
    }  
  
    // 비공개함수의 구현  
    .....  
  
    // 공개함수의 구현  
    .....  
} // End of class "AppController"
```

.....

```
public void run() {
```

```
this._pm = new PerformanceMeasurement();    // PerformanceMeasurement 객체를 생성
this.showMessage(MessageID.Notice_StartProgram);    // 프로그램 시작 메시지를 출력
this._pm.generateData();
this.showMessage(MessageID.Notice_SortedArrayStart);
this._pm.testSortedArrayBag();
this.showTestResults();
this.showMessage(MessageID.Notice_EndProgram);
```

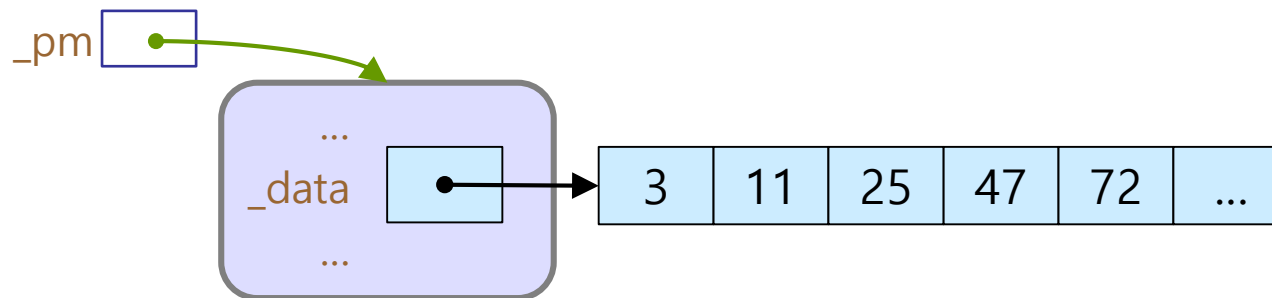
}



□ Class "AppController" 의 구현: run()

```
public class AppController {
    .....

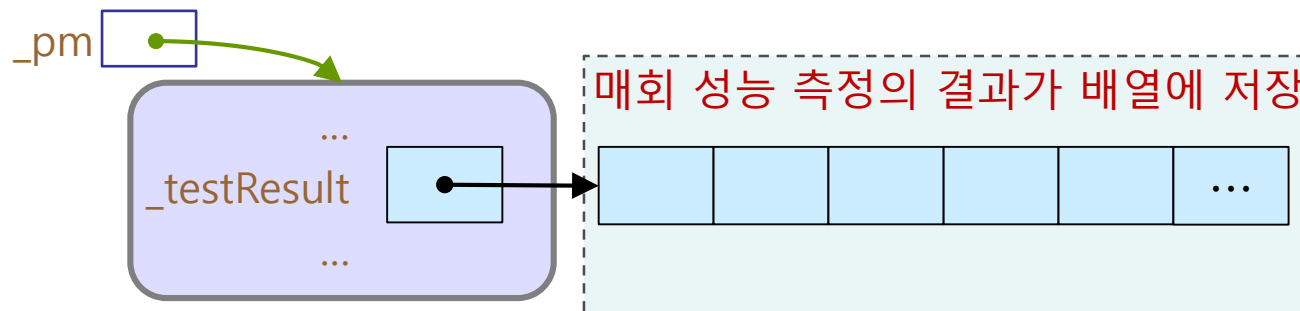
    // 공개함수의 구현
    public void run() {
        this._pm = new PerformanceMeasurement();
        this.showMessage(MessageID.Notice_StartProgram);
        this._pm.generateData();           // _pm 객체의 _data 배열에 난수를 생성하여 삽입
        this.showMessage(MessageID.Notice_UnsortedArrayStart);
        this._pm.testUnsortedArrayBag();
        this.showTestResults();
        this.showMessage(MessageID.Notice_EndProgram);
    }
}
```



□ Class "AppController" 의 구현: run()

```
public class AppController {
    .....

    // 공개함수의 구현
    public void run() {
        this._pm = new PerformanceMeasurement();
        this.showMessage(MessageID.Notice_StartProgram);
        this._pm.generateData();
        this.showMessage(MessageID.Notice_UnortedArrayStart); // 자료구조의 종류 출력
        this._pm.testUnsortedArrayBag();                     // 성능측정을 수행
        this.showTestResults();
        this.showMessage(MessageID.Notice_EndProgram);
    }
}
```



□ Class "AppController" 의 구현: run()

```
public class AppController {
```

```
.....
```

```
// 공개함수의 구현
```

```
public void run() {
```

```
    this._pm = new PerformanceMeasurement();
```

```
    this.showMessage(MessageID.Notice_StartProgram);
```

```
    this._pm.generateData();
```

```
    this.showMessage(MessageID.Notice_UnortedArrayStart);
```

```
    this._pm.testUnortedArrayBag();
```

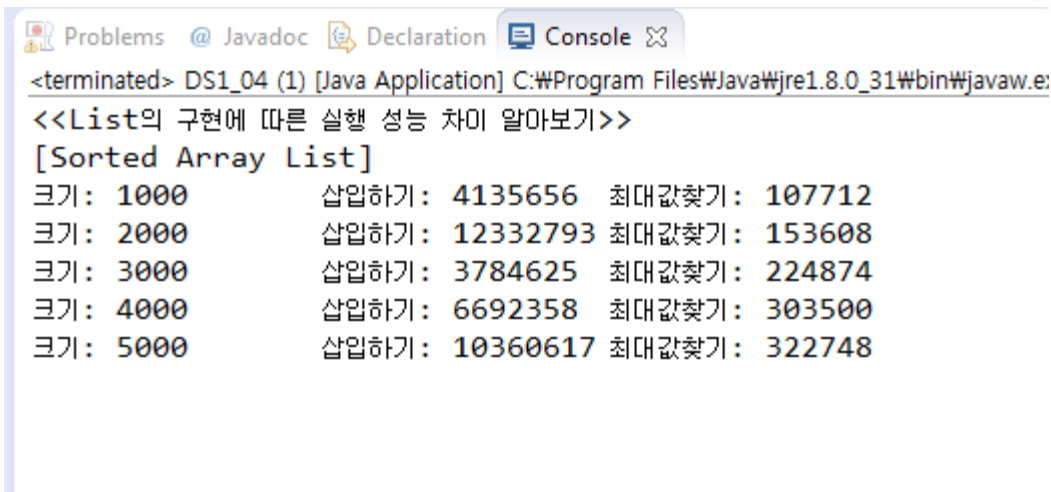
```
    this.showTestResults();
```

```
    this.showMessage(MessageID.Notice_EndProgram);
```

```
// 측정한 결과를 출력
```

```
// 종료 메시지를 출력
```

```
}
```



The screenshot shows a Java IDE console window with the following output:

```
<terminated> DS1_04 (1) [Java Application] C:\Program Files\Java\jre1.8.0_31\bin\javaw.e:
<<List의 구현에 따른 실행 성능 차이 알아보기>>
[Sorted Array List]
크기: 1000      삽입하기: 4135656   최대값찾기: 107712
크기: 2000      삽입하기: 12332793  최대값찾기: 153608
크기: 3000      삽입하기: 3784625   최대값찾기: 224874
크기: 4000      삽입하기: 6692358   최대값찾기: 303500
크기: 5000      삽입하기: 10360617  최대값찾기: 322748
```

□ Class "AppController"

```
public class AppController {  
    .....  
    // 비공개함수의 구현  
    private void showTestResults () {  
        TestResult testResults[] = this._pm.testResults () ;  
        for ( int index = 0; index < this._pm.numberOfTests () ; index++ ) {  
            this._appView.outputResult(  
                testResults[index].testSize(),  
                testResults[index].testInsertTime(),  
                testResults[index].testFindMaxTime() ) ;  
        }  
    }  
}
```

□ Class "AppController"

```
public class AppController {
    .....
    // 비공개함수의 구현
    private void showMessage(MessageID aMessageID)
    {
        switch(aMessageID) {
            case Notice_StartProgram:
                this._appView.outputMessage("<<List의 구현에 따른 실행 성능 차이 알아보기>>\n");
                break;
            case Notice_EndProgram:
                this._appView.outputMessage("<<성능 측정을 종료합니다>>\n");
                break;
            case Notice_UnsortedArrayStart:
                this._appView.outputMessage("[Unsorted Array List]\n");
                break;
            case Notice_SortedArrayStart:
                this._appView.outputMessage("[Sorted Array List]\n");
                break;
            case Notice_UnsortedLinkedStart:
                this._appView.outputMessage("[Unsorted Linked List]\n");
                break;
            case Notice_SortedLinkedStart:
                this._appView.outputMessage("[Sorted Linked List]\n");
                break;
            case Error_WrongMenu:
                this._appView.outputMessage("<<ERROR: 잘못된 메뉴입니다.>>\n");
                break;
            default:
                break;
        }
    }
}
```

Class "AppView"

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 구현

```
public class AppView {  
    // 비공개 상수/변수들  
  
    ....  
    // 생성자  
  
    .....  
    public void outputResult  
        (int aTestSize, long aTestInsertTime, long aTestFindMaxTime) { ... }  
  
    public void outputMessage (String aMessageString) { .... }
```


Enum "MessageID"

Enum "MessageID"

```
public enum MessageID {  
  
    // Message IDs for Notices:  
    Notice_StartProgram,  
    Notice_EndProgram,  
    Notice_UnsortedArrayStart,  
    Notice_SortedArrayStart,  
    Notice_UnsortedLinkedStart,  
    Notice_SortedLinkedStart,  
  
    // message IDs for Errors:  
    Error_WrongMenu,  
  
} //End of enum "MessageID"
```

Class "TestResult"



□ TestResult 객체의 구조

```
public class TestResult {  
    private int      _testSize;  
    private long     _testInsertTime;  
    private long     _testFindMaxTime;
```

□ TestResult 의 Member Functions

■ TestResult 의 Public Member function의 메소드

- 각 기능(생성자/getter/setter)에 맞도록 구현하세요.
- `public TestResult( )`
- `public TestResult(int aTestSize, long aTestInsertTime, long aTestFindMaxTime)`
- `public int testSize()`
- `public void setTestSize(int aTestSize)`
- `public long testInsertTime()`
- `public void setTestInsertTime(int aTestInsertTime)`
- `public long testFindMaxTime()`
- `public void setTestFindMaxTime(int aTestFindMaxTime)`

문제 자체를 위한 추상자료형(class): Performance Measurement



□ PerformanceMeasurement 의 속성(Attributes)

```
private static final int DEFAULT_NUMBER_OF_TESTS = 5 ;  
private static final int DEFAULT_FIRST_TEST_SIZE = 10000 ; // 첫 번째 실험 데이터 크기  
private static final int DEFAULT_SIZE_INCREMENT = 10000 ; // 실험 데이터 크기 증가량  
  
private int _numberOfTests ;  
private int _firstTestSize ;  
private int _sizeIncrement ;  
private int [] _testSizes ;  
private int [] _data ;  
private TestResult [] _testResults ;
```

□ PerformanceMeasurement의 공개함수

■ 생성자

- `public PerformanceMeasurement()`
- `public PerformanceMeasurement(
int givenNumberOfTests,
int givenFirstTestSize,
int givenSizeIncrement)`

■ `public int numberOfTests()`

- 현재 설정된 Test 횟수를 돌려준다.

■ `public int maxDataSize()`

- 실험 데이터의 최대 크기를 계산하여 돌려준다.
- 계산 방법: `this._firstTestSize + this._sizeIncrement * (this._numberOfTests - 1)`

■ `public void generateData ()`

- 성능 측정에 필요한 데이터를 생성한다.

■ `public TestResult[] testResults ()`

- 성능 측정 결과를 반환한다.

PerformanceMeasurement의 공개함수 성능 비교

- `public void testUnsortedArrayBag()`
 - Unsorted Array로 구현한 List의 성능을 측정한다.
- `public void testSortedArrayBag()`
 - Sorted Array로 구현한 List의 성능을 측정한다.
- `public void testUnsortedLinkedBag()`
 - Unsorted Linked List로 구현한 List의 성능을 측정한다.
- `Public void testSortedLinkedBag()`
 - Sorted Linked List로 구현한 List의 성능을 측정한다.

□ PerformanceMeasurement : 생성자()

```
public PerformanceMeasurement() {  
    this._numberOfTests = PerformanceMeasurement.DEAULT_NUMBER_OF_TESTS ;  
    this._firstTestSize = PerformanceMeasurement.DEFAULT_FIRST_TEST_SIZE ;  
    this._sizeIncrement = PerformanceMeasurement.DEFAULT_SIZE_INCREMENT ;  
  
    this._data = new int[this.maxDataSize()] ;  
    this._testResults =  
        new TestResult[PerformanceMeasurement.DEFAULT_NUMBER_OF_TESTS] ;  
}
```

실험 데이터 생성

□ 난수를 이용하자 !

■ 난수 (Random Number)

- 아무 규칙 없이, 무작위적으로 얻어지는 수
- 하나의 난수를 얻고 나서, 그 다음 난수를 얻을 때 이전 난수와 어떠한 연관관계도 없이 무작위적으로 얻어져야 한다.

□ 난수를 이용한 실험 데이터 생성 [O]

```
import java.util.*;
```

```
.....
```

```
Random random = new Random();
```

```
int data[MaxSize];
```

```
int i = 0 ;
```

```
while ( i < MaxSize ) {
```

```
    data[i] = random.nextInt( MAX_SIZE );
```

```
    i++ ;
```

```
}
```

```
.....
```

실험 데이터를 무작위로
MaxSize만큼 생성하여
배열 "data[]"에
저장하려고 한다.

□ 난수를 이용한 실험 데이터 생성 [4]

```
import java.util.*;
```

```
.....
```

```
Random random = new Random();
```

```
int data[MaxSize];
```

```
int i = 0 ;
```

난수 생성을 하는 객체를 생성한다.

```
while ( i < MaxSize ) {
```

```
    data[i] = random.nextInt( MAX_SIZE );
```

```
    i++ ;
```

```
}
```

```
.....
```

"nextInt()"

정수형 난수를
얻는다.

"MAX_SIZE"

0~(Maxsize-1) 사이의
정수를 얻게 해 준다.

□ 유사 난수란?

■ 난수 (Random Number)

- 아무 규칙 없이, 무작위적으로 얻어지는 수
- 하나의 난수를 얻고 나서 그 다음 난수를 얻을 때, 이전 난수와 어떠한 연관관계도 없이 무작위적으로 얻어져야 한다.

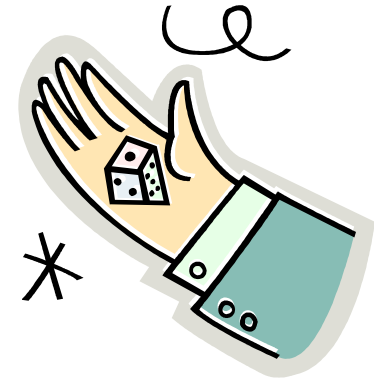
■ 유사 난수 (Pseudo Random Number)

- 엄격히 말해서, 난수는 계속 무작위적으로 수가 얻어져야 한다.
- `nextInt ()`를 이용하여 계속 무작위적으로 다음 수를 얻을 수 있을까?

□ 난수 대신 유사난수 ?

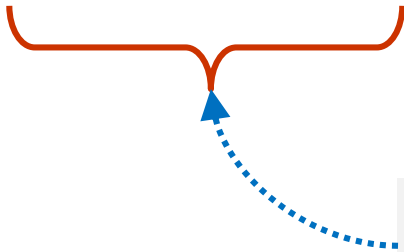
■ 난수

- 5, 2, 1, 4, 6, 3, 1, 5, 6, 3, 4, 2,

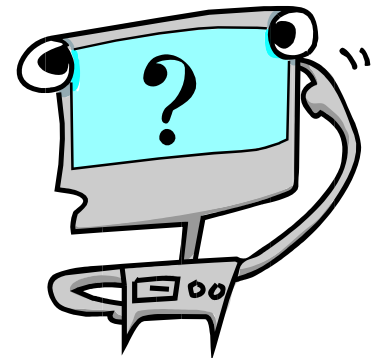


■ 유사난수

- 5, 2, 1, 4, 6, 3, 5, 2, 1, 4, 6, 3, 5, 2, 1, 4, 6, 3,



반복되는 sequence가
매우 길다면?



□ PerformanceMeasurement : 공개함수

```
public void generateData (){  
    // 성능 측정에 필요한 데이터를 생성한다.  
    int i = 0;  
    Random random = new Random();  
    while ( i < this._maxTestSize ) {  
        this._data[i] = random.nextInt (this._maxTestSize) ;  
        i++;  
    }  
}  
  
public int numberOfTests() {  
    //현재 설정된 Test 횟수를 반환한다.  
    return this._numberOfTests;  
}
```

□ 비공개 함수

```
private int maxCoinValue (UnsortedArrayBag<Coin> aCoinBag)
{
    int maxValue = 0 ;
    for (int i = 0; i < this._coinBag.size() ; i++) {
        if ( maxValue < aCoinBag.elementAt(i).value() ) {
            maxValue = aCoinBag.elementAt(i).value() ;
        }
    }
    return  maxValue ;
}
```

□ PerformanceMeasurement : testSortedArrayBag()

```

public void testSortedArrayBag() {
    // Sorted Array로 구현한 List의 성능을 측정한다.
    UnsortedArrayBag<Coin> bag;
    Coin maxCoin;
    int testSize;
    long timeForAdd, timeForMax;
    long start, stop;
    int testDataCount ;
    int testCount = 0;

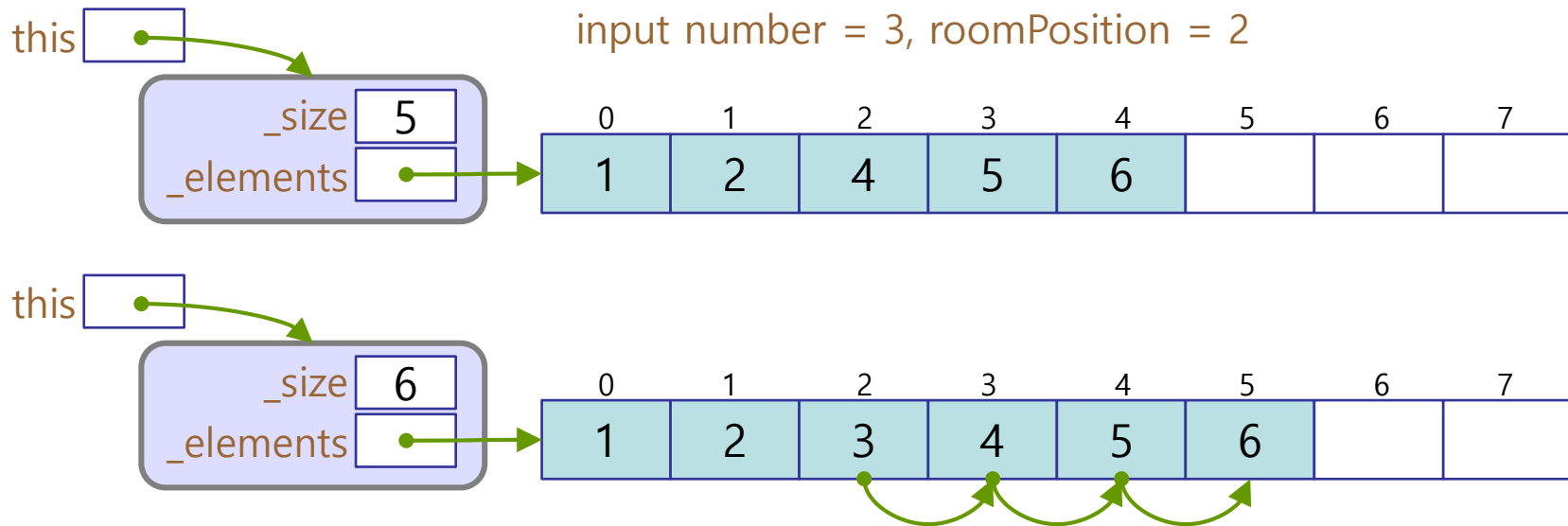
    testSize = this._firstTestSize ;
    while (testCount < this._numberOfTests) {
        testSize = this._testSizes[testCount];
        bag = new UnsortedArrayBag<Coin>(testSize);
        testDataCount = 0;
        timeForAdd = 0;
        timeForMax = 0;
        while (testDataCount < testSize) {
            Coin coin = new Coin(this._data[testDataCount]);
            start = System.nanoTime();
            bag.add(coin);
            stop = System.nanoTime();
            timeForAdd += (stop - start);

            start = System.nanoTime();
            maxCoin = this.maxCoinValue (bag);
            stop = System.nanoTime();
            timeForMax += (stop - start);
            testDataCount ++;
        }
        this._testResults[testCount] = new TestResult(testSize, timeForAdd, timeForMax);
        testSize += this._sizeIncrement ;
        testCount++;
    }
}

```

☐ SortedArrayBag Class의 삽입

- 배열의 sort 순서에 맞는 위치를 찾아 삽입한다.
 - 먼저, 순서에 맞는 위치를 찾는다.
 - 그 다음, 뒤에서부터 찾은 위치까지 하나씩 뒤로 미루어 자리를 만든다.
 - 찾은 자리에 새로 삽입할 값을 입력한다.
- SortedArrayBag의 `_elements[]` 초기화 크기는 `maxSize` 가 맞을까?



□ 요약과 질문

■ 실험 준비와 측정

- 난수를 이용한 데이터 생성 방법을 이해한다.
- 시간 측정 방법을 이해한다.

■ 각각의 기능에 대해, 데이터의 크기에 따른 시간의 변화를 이해한다.

■ 위에서 언급한 사항에 대해, 자신의 실험 결과를 보고 각자의 의견을 논하시오.

실험 크기 변경하여 해보기

□ PerformanceMeasurement 의 속성(Attributes)

```
private static final int DEFAULT_NUMBER_OF_TESTS = 10 ;  
private static final int DEFAULT_FIRST_TEST_SIZE = 100000 ; // 첫 번째 실험 데이터 크기  
private static final int DEFAULT_SIZE_INCREMENT = 100000 ; // 실험 데이터 크기 증가량
```

```
private int _numberOfTests ;  
private int _firstTestSize ;  
private int _sizeIncrement ;  
private int [] _testSizes ;  
private int [] _data ;  
private TestResult [] _testResults ;
```

보고서 작성과 제출

□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제04주] 가방 성능 비교
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서

1. 주요 알고리즘 /자료구조 /기타
2. 함수 설명서
3. 종합 설명서

2. 프로그램 장단점 분석

3. 실행 결과 분석

1. 입력과 출력 (화면 capture 시킬 것)
2. 결과 분석

4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

- DS1_04_학번_이름.zip

- 폴더의 구성

- ◆ DS1_04_학번_이름

- 프로그램

- 프로젝트 폴더 / 소스

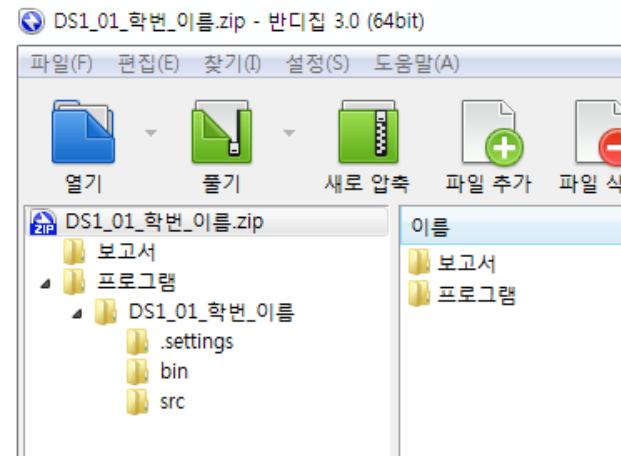
- 메인 클래스 이름 : DS1_04_학번_이름.java

- 보고서

- 이곳에 보고서 문서 파일을 저장한다.

- 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

- 문서는 pdf 파일로 만들어 제출한다.



[제 4 주] 실습 끝



